

Silent-Failure Audit

Triggered by the duplicate Worker ID incident. This document contains the fix for that exact bug, and then every other place in the product where the same **class** of mistake is waiting to happen: a rule the user cannot see, enforced too late, reported in a way they cannot act on.

Each item is a **ready-to-assign ticket** — title and short description, no file paths, no prescribed solution. Under most of them is a shaded **background note** with the diagnosis and the exact location. That note is for you, not for the dev. If a diagnosis in it turns out to be wrong, the ticket above it is still correct.

25 tickets · 4 must-fix · audited against `dev` on all three repos · 6 August 2026

Backend `3d8248e` · Client panel `18dcf7af` · Mobile `507d3880`

Out of scope by request: Wergonic Admin, landing page, and the client-panel prototype routes.

WHAT ACTUALLY HAPPENED

Three separate faults lined up to produce one dead end.

The client typed Worker ID `1` and tapped Next. **First**, Worker ID is unique across the entire database rather than per company, so `1` was already taken — by another customer, whose data they cannot see. **Second**, the backend replied with a perfectly clear message, and the mobile app threw it away and showed the word “Bad request” instead. **Third**, nothing checked the ID while they were typing it, so they only found out after committing.

None of the three is hard to fix. What made it expensive is that all three are systemic — the same three patterns repeat across the product, and that is what the rest of this document is about.

READ THIS BEFORE ASSIGNING

Four issues here are worse than the one the client reported.

A1 lets one customer attach another customer's workers to their sessions. **A2** is silently corrupting stored worker data on every single mobile session, today, for every customer. **A4** means any worker invited from the web has never been able to record a session. These were not reported by anyone — they are quiet, which is exactly why they lasted.

Priority summary

Ordered by what it costs to leave alone. P0 is customer-visible or customer-affecting right now.

ID	PRIORITY	WHERE	TICKET
----	----------	-------	--------

DO FIRST — THE CLIENT'S BUG, AND THREE QUIETER ONES THAT ARE WORSE

A0	P0	BACKEND MOBILE	Fix duplicate Worker ID producing a dead end — the reported bug
A1	P0	BACKEND	Fix sessions being able to use another company's workers
A2	P0	MOBILE BACKEND	Fix every mobile session overwriting the worker's gender and dominant arm

ID	PRIORITY	WHERE	TICKET
A3	P0	MOBILE	Fix a recorded session getting stranded when the worker cannot be created
A4	P0	WEB BACKEND	Fix workers invited from the web never getting a worker profile
HIGH — THE USER CANNOT SEE OR ACT ON WHAT WENT WRONG			
B1	P1	BACKEND	Fix user mistakes being returned as “Something went wrong”
B2	P1	MOBILE	Fix the mobile app hiding every error message the backend sends
B3	P1	WEB	Fix the client panel dropping per-field validation messages
B4	P1	BACKEND	Fix work categories and labels being creatable inside another company
B5	P1	BACKEND	Fix changing a user's email to one that already exists breaking the account
B6	P1	MOBILE	Fix finishing a session before the recording has finished saving
B7	P1	BACKEND	Fix registering the same device twice at once returning a server error
MEDIUM — CONFUSING, BUT THE USER CAN WORK AROUND IT			
C1	P2	MOBILE	Fix mobile fields accepting values the backend will reject
C2	P2	WEB	Fix client panel fields accepting values the backend will reject
C3	P2	BACKEND	Fix over-long values crashing profile creation and Excel import
C4	P2	BACKEND	Fix duplicate label, operator and report-template names being allowed
C5	P2	MOBILE	Fix the work library picker closing as if the selection saved when it failed
C6	P2	BACKEND	Fix a deactivated member being impossible to re-invite
C7	P2	BACKEND	Fix bulk archive-settings update silently skipping what it cannot change
C8	P2	WEB	Fix duplicate factory name showing no error on the field
C9	P2	WEB	Fix the invite form accepting an invalid email address
C10	P2	BACKEND	Fix the session export revealing whether another company's session exists
LOW — CLEANUP, AND ONE DECISION FOR YOU			
D1	P3	BACKEND	Check whether two companies are allowed to have the same name
D2	P3	BACKEND MOBILE	Check the leftover first name / last name fields on worker profiles
D3	P3	BACKEND	Fix firmware update returning the wrong status code for a missing record
D4	P3	MOBILE	Fix the “no organization” error being English-only

Do first

A0 is the bug the client reported. A1 to A4 were found by this audit and are, in my judgement, more damaging.

A0 Fix duplicate Worker ID producing a dead end BACKEND MOBILE

A user creating a worker types a Worker ID that already exists. Nothing warns them while typing, and when they continue they get an error saying only “Bad request”, with no way to understand or fix it. Four things need to change. **One:** Worker IDs must be unique per organization, not across the whole database — today a customer can be blocked by an ID belonging to a company they cannot see, and the error message confirms that company's record exists. **Two:** the mobile app must show the message the backend actually sent instead of its own generic text — this applies everywhere, not just this screen. **Three:** the Worker ID field must check availability as the user leaves it and show a red message under the field, blocking Next. **Four:** when the worker is only created at the end of a session, the ID must be validated at the start instead, so a finished recording can never be rejected.

BACKGROUND — FOR AMINE

Point one is a one-line model change plus a migration: `worker_id` is declared `unique=True`, and both create endpoints check `Profile.objects.filter(worker_id=...)` with no organization filter. It needs to become a unique-together on worker id + organization, and the two lookups need the org added.

Point three is smaller than it sounds — the backend already has this exact pattern for other fields:

`/users/email/check/` and `/users/personal_number/check/` are availability endpoints the client panel already calls while the user types. Adding `/users/worker_id/check/`, scoped to the active org, and wiring it into both apps is the whole job.

Points two and four are covered in more depth by B2 and A3 — if you assign those, this ticket only needs points one and three.

`users/models/baseUserModel.py:205` · `organizations/views/orgUserProfileViews.py:96, 281` · `lib/core/errors/api_exceptions.dart:157`

A1 Fix sessions being able to use another company's workers BACKEND

When a session is created, the backend looks up the worker and the ergonomist by their ID numbers and never checks that they belong to the company making the request. Anyone with a valid login can attach a worker or an ergonomist from a different customer to their own session, just by sending a different number. The lookups must be restricted to the organization the request belongs to, and return “not found” otherwise.

BACKGROUND — FOR AMINE

This is the same root mistake as the Worker ID bug, pointed the other way. The Worker ID rule was scoped too wide; here the lookup is not scoped at all. Both come from the same habit of fetching a record by primary key and trusting the number.

There are five unscoped lookups in the one function. The request already resolves the caller's organization a few lines above, so the fix is mechanical — but it needs a test, because nothing currently proves it. **I have not checked whether this has ever actually been exploited;** I am reporting that the code allows it.

`user_sessions/views/session_views.py:216, 218, 230, 231, 249`

A2 Fix every mobile session overwriting the worker's gender and dominant

arm **MOBILE** **BACKEND**

Every time a session is started from the mobile app, the app sends the worker's gender and dominant arm in the wrong format, and the backend writes whatever it receives straight onto the worker's profile without checking it. The stored value ends up being something the system does not recognise as a valid gender or arm. This happens on every session, for every worker, and nobody is told. Fix the format the app sends, and make the backend reject values that are not one of the allowed options instead of saving them.

BACKGROUND — FOR AMINE

Concretely: the app sends `female` and `Right`; the system's allowed values are `FEMALE` and `RIGHT`. Django only enforces the allowed list on forms, not on direct assignment, and the helper that writes the profile assigns the raw string and saves. So the profile row now holds a value outside its own allowed set.

Two things make this worse than it looks. It runs on **session start, every time** — so it re-corrupts a worker every session, including workers whose details were entered correctly from the web. And the same helper also overwrites weight, age and resting heart rate from the session payload, so mobile silently wins over anything an admin edited in the client panel.

What I could not determine: whether gender or dominant arm feed into any ergonomic calculation. If they do, this has been quietly affecting results. Worth confirming with Farhad before deciding how far back it matters. I was not able to check the production database to measure how many rows are affected.

```
lib/core/enums.dart:16, 63 · lib/features/sessions/session/data/models/work_session/work_session_model.dart:91  
· organizations/views/orgProfileUtils.py (CreateProfile) · user_sessions/views/session_views.py:266
```

A3 Fix a recorded session getting stranded when the worker cannot be created **MOBILE**

When a user creates a new worker while offline, the app moves straight on without creating that worker. The worker is only created much later, at the end, when the session is synced. If the backend rejects it at that point — a duplicate ID, a value too long, anything — the recording is already done and there is no way to correct the worker details or try again. The only button is Close. Validate the worker as early as the connection allows, and when creation does fail at sync time, let the user edit the worker details and retry without losing the recording.

BACKGROUND — FOR AMINE

This is the mechanism that turned the client's small mistake into a real loss. Typing a duplicate ID is recoverable in two seconds if you are told immediately. Being told after a full recording session, with no edit and no retry, is what made it look broken.

The offline branch skips profile creation entirely and navigates forward; the summary screen then attempts the creation as the first step of sync. The recovery path is the important half of this ticket — even with A0 fixed, some other validation error will eventually land here.

```
create_or_update_user_info_for_session.page_view.dart:287-320 · session_summary.page.dart:144 ·  
sync_progress_stepper.widget.dart:132
```

A4 **Fix workers invited from the web never getting a worker profile** WEB BACKEND

Adding a worker from the Users page of the client panel creates their login and sends the invitation, but never creates their worker profile — the form does not even ask for a Worker ID, which is now required. When that worker logs into the mobile app and starts a session, the request fails with a server error. Either collect the Worker ID in that form and create the profile alongside the account, or stop offering the worker role there and point admins at the Workers page, which does it correctly.

BACKGROUND — FOR AMINE

This is fallout from the worker-naming refactor. The Workers page was updated to collect a Worker ID; the older “Add new user” flow on the Users page was not, and it is the one that can invite a worker with a login.

The failure at the other end is ugly: the session endpoint looks up the worker's profile, does not find one, and the exception is caught by a catch-all that returns a generic 500. So the worker sees a server error with no explanation — the same pattern as B1.

Worth asking the team which of the two paths is meant to be the real one. Having two different ways to add a worker, one of which is broken, is the actual problem.

apps/client-panel/src/features/users/components/TableControls/AddNewUserButton/Form2/Form2.tsx · organizations/views/org_user/list_create_delete.py · user_sessions/views/session_views.py:231

High

These are the reason a small mistake becomes a support ticket. In each one the system knows what went wrong and does not tell the person who can fix it.

B1 **Fix user mistakes being returned as “Something went wrong”** BACKEND

Several endpoints wrap everything they do in a single catch-all that returns a generic server error. A duplicate value, a missing required field or an over-long name all come back looking identical to a genuine crash, so no client app can show anything useful. Separate the two: things the user can fix return a proper validation error with a specific message, and only real failures return a server error.

BACKGROUND — FOR AMINE

Affects session creation, session import, firmware create and update, and organization creation. Session import is the one that matters most — an import failing with no reason at all is a support call every time.

This also hides the real error rate from Sentry, because everything is logged as one generic exception.

user_sessions/views/session_views.py:348 · user_sessions/views/session_import_view.py:184 · devices/views.py:171, 265 · organizations/views/organizationView.py:242

B2 Fix the mobile app hiding every error message the backend sends MOBILE

The app reads the explanation the backend sends with an error, stores it, and then never shows it. Instead it displays its own fixed text — “Bad request”, “Not found”, “Internal server error” — regardless of what actually happened. Every clear message the backend produces is discarded before the user sees it. Show the backend's message when there is one, and fall back to the generic text only when there isn't.

BACKGROUND — FOR AMINE

This is the single change that would have prevented the client's incident from being a mystery. The backend was already returning “Worker ID “1” is already in use.” — the app had that string in memory and chose to print “Bad request”.

It is a small change in one file, and it improves every error screen in the app at once. If you only fund one ticket in this section, fund this one.

```
lib/core/errors/api_exceptions.dart:145-160 (the context value is never used by showMessage) ·  
lib/core/errors/error_mappers.dart
```

B3 Fix the client panel dropping per-field validation messages WEB

When the backend rejects a form and says which field was wrong, the client panel only looks for a single general message and ignores the per-field ones. The user gets a vague toast, or in some cases an empty one, and no indication of which field to correct. Handle all the shapes the backend replies in, through one shared helper, and show the message against the field it belongs to.

BACKGROUND — FOR AMINE

Same fault as B2, different app: the information arrives and is thrown away. It shows up across workers, team, assessments, measurements, work library and the shared table components.

Note the dependency — B1 decides what shape the backend sends. Doing B3 first means the team will guess at shapes that are about to change. Assign B1 first, or assign them together.

```
apps/client-panel/src/utils/parseApiError.ts · hooks/useResourceMutation.ts · features/workers/hooks.ts ·  
features/team/hooks.ts
```

B4 Fix work categories and labels being creatable inside another company BACKEND

Creating a work category takes the company from the address in the request and never checks that the person making it belongs to that company. Creating a label looks up its parent category without checking either, and the permission check that was meant to guard session events is commented out. Any logged-in user can create records inside another customer's data. Verify the caller's organization on every create.

BACKGROUND — FOR AMINE

Same family as A1. The commented-out permission check is worth asking about specifically — someone disabled it deliberately at some point, and it was never put back.

```
work_tags/views.py:58, 108, 193
```

B5 Fix changing a user's email to one that already exists breaking the account BACKEND

When an admin changes a user's email address to one already in use, the backend saves it first and only afterwards tries to update the login provider, which refuses. The result is an account whose email in our database no longer matches the one it can actually log in with. Check the address is free before saving, and reject it cleanly if it is not.

BACKGROUND — FOR AMINE

Recovering one of these by hand is unpleasant, and the admin who caused it just sees a server error. Same check-before-write pattern as A0 — the constraint exists, it is simply enforced too late.

`organizations/views/org_user/update.py:52, 84` · `users/services/user_service.py:56`

B6 Fix finishing a session before the recording has finished saving MOBILE

When a user ends a session, the app asks the recording and the data file to stop, waits ten seconds, and moves on whether or not they succeeded. A failure is logged for us but the user is taken to the summary as if everything worked, with numbers that may be incomplete. If a step fails or times out, tell the user and offer to retry rather than presenting partial results as final.

BACKGROUND — FOR AMINE

This is the highest-consequence version of the pattern in this document: the user is shown a successful outcome for something that failed. It would also explain sessions with numbers that look wrong for no clear reason — **speculation on my part**, but it is a plausible source and cheap to rule out.

`work_session_recording.page.dart:306, 318, 332, 345`

B7 Fix registering the same device twice at once returning a server error BACKEND

Device and sensor registration checks whether the serial number already exists and then creates the record. If the same serial is submitted twice at the same moment, both checks pass and the second create fails with a server error instead of a clear message. Handle the collision and return a proper “already registered” response.

BACKGROUND — FOR AMINE

Low likelihood on its own — it needs two submissions in the same instant. Worth doing because it is the same check-then-create shape as A0 and B5, so it will be in the developer's head anyway. Do not let anyone spend a day on it.

`devices/serializers.py:28, 32`

Medium

Real, reproducible, and worth a sprint of cleanup — but the customer can get past all of these.

C1 Fix mobile fields accepting values the backend will reject MOBILE

Worker ID, workplace, comment, age, weight and resting heart rate have no length or range limits on mobile. The user can type a value the backend cannot store, and the only feedback is a generic error after submitting. Add limits matching the backend, and sensible ranges for the numeric fields, shown as the user types.

C2 Fix client panel fields accepting values the backend will reject WEB

Organization name, support ticket subject and work cycle task name have no length limits in the client panel. On the organization form the edit mode also closes when the save fails, so the user loses what they typed. Add the limits and keep the form open on failure.

C3 Fix over-long values crashing profile creation and Excel import BACKEND

Worker profile creation and the work-cycle Excel import both pass values straight to the database without checking their length. Anything too long produces a database error that surfaces as a generic failure. For the import in particular, tell the user which row and which column was too long.

C4 Fix duplicate label, operator and report-template names being allowed BACKEND

Nothing prevents two labels with the same name in the same category, two operators with the same name on the same workstation, or two report templates with the same name for the same user. The user then sees two identical entries in a dropdown with no way to tell them apart. Add the missing uniqueness rules — scoped to the parent, not globally.

BACKGROUND — FOR AMINE

Note the contrast with A0: there the rule was too wide, here there is no rule at all. Both come from nobody deciding what the identity of these things actually is. Worth settling once as a convention.

`work_tags/models.py:74` · `workcycles/models.py:54` · `users/models/ReportTemplateBasedModel.py:7`

C5 Fix the work library picker closing as if the selection saved when it failed MOBILE

Picking an activity from the work library closes the picker immediately, whether or not the selection was actually saved. If it failed, the user returns to the recording screen believing they chose something. Keep the picker open and show the error when the save fails. The same applies to the session conditions screen, where a failure appears only as a brief message that disappears on its own.

C6 Fix a deactivated member being impossible to re-invite BACKEND

Once a member is deactivated, inviting them again does not reactivate them. The invite is refused with a message telling the admin to reactivate the person from the team list, and there is no reactivation control there. The person is stuck. Either make accepting an invitation clear the deactivated flag, or build the reactivation control the message refers to.

BACKGROUND — FOR AMINE

Related to the invitation flaws already parked from the earlier review. If you are picking that work up, do them together rather than separately.

`organizations/views/org_user/list_create_delete.py:202` · `organizations/views/invitationView.py:219, 240`

C7 Fix bulk archive-settings update silently skipping what it cannot change BACKEND

Updating several archive settings at once quietly ignores any that do not belong to the caller and still reports success. The user believes all their changes were applied. Report back which ones were updated and which were skipped.

C8 Fix duplicate factory name showing no error on the field WEB

Adding a factory with a name that already exists is rejected by the backend, but the form shows nothing against the name field — the dialog just stays open. Show the error on the field, and check availability before submitting.

C9 Fix the invite form accepting an invalid email address WEB

The team invite form sends whatever is typed without checking it is a valid email address. The backend rejects it and the user sees a message with no explanation attached. Validate the address in the form before sending.

C10 Fix the session export revealing whether another company's session exists BACKEND

The statistics export looks up the session first and checks ownership afterwards, so it answers “forbidden” for a session that exists in another company and “not found” for one that does not. That difference lets an outsider work out which session numbers are real. Look the session up within the caller's company and return “not found” either way.

Low, and one decision

D1 Check whether two companies are allowed to have the same name BACKEND

Company names are currently unique across the entire system, so the second company called “Nordic Logistics” to sign up is refused with “This company name is already taken”. This may be intentional. Decide: if duplicates should be allowed, remove the rule and show something else alongside the name where it matters; if not, close this and leave it as is.

BACKGROUND — FOR AMINE

This is your decision, not a developer's — that is why it is phrased as a question. It is the same shape as the Worker ID rule, so it is worth answering deliberately rather than by accident.

`users/models/organizationModel.py:52` · `organizations/views/referralViews.py:168`

D2 Check the leftover first name / last name fields on worker profiles BACKEND MOBILE

Worker profiles no longer have a first name and last name — that change was made deliberately when Worker ID was introduced. Both apps still send those fields, the backend still ignores them, and the mobile app still splits the Worker ID into a pretend first and last name for display. Nothing is lost today, but it is confusing to read. Decide the intended model and remove what is left over.

BACKGROUND — FOR AMINE

The automated audit initially flagged this as a high-severity data-loss bug. It is not — I checked, the same value is also sent as the Worker ID, so nothing the user types is dropped. It is cleanup, and I have graded it accordingly.

`create_profile_data.model.dart:82, 85` · `apps/client-panel/src/api/organizations.ts:56` · `organizations/views/orgUserProfileViews.py:102`

D3 Fix firmware update returning the wrong status code for a missing record BACKEND

Updating a firmware record that does not exist returns a validation error instead of “not found”. Correct the status code.

D4 Fix the “no organization” error being English-only MOBILE

Starting a session without an organization shows an error written in English regardless of the app language. Everything else in the app is translated. Move it into the translation files.

Why this keeps happening

Five habits produced every item in this document. Fixing the twenty-five tickets without changing these will produce a new list in three months.

1 · RULES ARE ENFORCED AT THE LAST POSSIBLE MOMENT

The database is being used as the validator.

Length limits, allowed values and uniqueness are mostly enforced only when the row is written. By then the user is several steps past the field they got wrong, and the failure arrives as an exception rather than as advice. Every limit that exists on a database column should also exist on the form and in the serializer.

2 · ERROR MESSAGES ARE TREATED AS NOISE

Both apps receive good explanations and print their own instead.

The backend often knows exactly what is wrong. The mobile app discards that text on every error. The client panel keeps only the general part and drops the per-field part. And some backend endpoints destroy the explanation before it is ever sent, by catching everything as one generic failure. One agreed error shape, one place that reads it, in each app.

3 · AN ID NUMBER IS TRUSTED AS PROOF OF OWNERSHIP

Records are fetched by number without asking whose they are.

Sessions, categories, labels and exports all fetch by ID and assume the caller is entitled to it. A helper that returns the caller's organization already exists and is used in some places and not others. Any lookup on a customer-owned table should go through it — and that is a review checklist item, not a one-off fix.

4 · THE SCREEN ADVANCES BEFORE THE WORK IS CONFIRMED

Navigation happens first, the result is checked afterwards, if at all.

The worker form, the work library picker and the end of a recording all move the user forward before knowing whether the operation succeeded. When it fails, the user is somewhere else, and sometimes past the point where correcting it is possible. Nothing should navigate on the assumption of success.

5 · NOBODY OWNS THE SHAPE OF THE DATA BETWEEN THE THREE APPS

Two apps and a backend disagree about the same fields, quietly.

Gender arrives in a format the backend does not recognise and is stored anyway. Name fields are still sent to a place that deleted them. Two different screens create a worker and only one of them collects the required ID. None of this produces an error — it produces wrong data and broken accounts. This is the one that needs a decision from you rather than a ticket: what is the agreed shape, and who checks it.

How this was checked

So you know what the confidence level actually is.

- All four repositories were pulled first; the audit ran against **dev** on backend, client panel and mobile.

- **Ten parallel passes** over the three codebases produced 74 candidate issues, each then challenged by a second reviewer whose instructions were to disprove it.
- Survivors went through a **second evidence pass** requiring the actual line of code to be quoted. Seven were thrown out, the rest merged down to the 25 tickets here.
- I personally re-checked all four P0 items and re-graded two that the automated passes got wrong — **A2 was under-reported** and **D2 was over-reported**.
- **Not verified:** nothing was reproduced against a running system, and I was not able to query production, so the number of affected records for A2 is unknown. Wergonic Admin, the landing page and the prototype routes were excluded as instructed.